

ORIE 4580: Modeling LLM-Query Serving Systems

Anay Bhat, Tanvi Kigga, Sanya Kohli, Hana Mariappa, Dylan Shah

Executive Summary

This project investigates the critical performance bottlenecks in Large Language Model (LLM) serving systems, specifically addressing the resource contention between compute-heavy prompt processing ("prefill") and memory-bound text generation ("decode"). By developing a discrete-event simulation calibrated to Llama-3-70B hardware profiles, we quantified the trade-offs between three distinct scheduling policies: prefill-prioritizing, decode-prioritizing, and chunked-refill. Our analysis reveals a fundamental, rigid inverse relationship between initial responsiveness and streaming smoothness. The baseline prefill-prioritizing policy excels at minimizing Time to First Token (TTFT), maintaining a near-zero violation rate even at high loads. However, this comes at the cost of user experience as large incoming requests effectively "freeze" the GPU for active users, causing "generation stalls" that drive Time Between Tokens (TBT) violations to nearly 18%.

In contrast, the chunked-refill strategy, which implements the "stall-free" scheduling methodology proposed in [Agrawal et al., 2024](#), successfully eliminates these generation stalls by breaking large jobs into smaller, non-blocking iterations. Our optimization analysis identified 10-token chunks as the ideal parameter to minimize overhead. While this approach stabilizes the streaming experience and keeps TBT violations near the 5% target, it imposes a significant latency tax consistent with findings in the research paper causing TTFT violation rates to rise significantly under saturation, exceeding 70%.

Consequently, our findings demonstrate that there is no single "best" scheduling policy. Instead, the optimal choice depends entirely on the specific application requirements. For interactive workloads like chatbots, we recommend chunked-refill, as human users perceive high TBT as a system failure, whereas a slightly longer initial wait (higher TTFT) remains tolerable. Conversely, for offline or batch processing tasks where no human is watching the stream, prefill-prioritizing is the superior choice to maximize aggregate throughput and minimize total completion time. This simulation framework therefore serves as a critical decision-support tool,

allowing platform engineers to select and tune the scheduling logic that best aligns with their specific service-level objectives.

Modeling Approach

To start, we are building a computer simulation to understand how LLM systems such as ChatGPT decide which user inputs to process first, which to batch together, and how these choices affect speed, cost, and user experience. When a user submits a query, the model processes it in two distinct stages: prefill and decode, which behave very differently on the GPU.

The prefill stage is the setup phase, during which the model reads and processes the entire user prompt at once. This step is computationally intensive and memory-intensive, as it requires the GPU to process each word. Because the prefill costs a large amount of GPU, it can become the bottleneck and cause a large queue. If too many queries arrive, the system has to decide to either start the prefill or delay to avoid disrupting the ongoing decode. Decode is the response generation stage, when a model generates the response token by token. Where each token depends on the previous one, so decoding must happen in order within a single query, but multiple queries can run in parallel. Each of the decode steps is relatively fast.

Two important metrics we considered were Time to First Token (TFT) and Time Between Tokens (TBT). TTFT measures how long a user has to wait before anything appears, while TBT measures the average delay between successive output tokens during decode. As the arrival rate increases, more queries arrive and the queue grows, and batches get larger. For the service level objective (SLO), we are looking at 95% of queries having $TTFT \leq 500$ ms and 95% of queries having a $TBT \leq 50$ ms.

We are also considering steady-state considerations, as for each of our functions, we are running the simulations at least 10,000 queries, and discarding the first 2,000 queries as a warmup to reduce the initialization bias.

Scheduling Policies

We are analyzing three different policies: prefill prioritization (FCFS baseline), decode prioritization, and chunked refill.

Prefill prioritization means that new incoming queries are quickly admitted into the system so the queue before the service is. This will improve the Time to First Token (TTFT) for

new users; however, it may slow down the queries already in decode, as they will pause. It will admit as many first-come, first-served requests as possible, up to the maximum batch size, maximizing throughput.

The decode prioritizing implies that the system will finish generating the response for the query it is working on before working on new queries. Any request in the currently decode phase, no new work is admitted. This will imply lower Time Between Tokens (TBT), and may lead to higher TTFT for newly arriving users. From the user perspective, decode prioritization offers a smoother experience as it does not stop and pause while generating a response.

The chunked refill policy is a combination policy aimed at balancing the goal of prefill and decode prioritization. This will find the balance between TTFT and TBT. The system will run a large batch until some of the queue is shortened. Instead of running a full batch like in prefill, or a minimal batch with a fixed number of requests, it creates a pattern to follow fast decoding windows.

Model Details

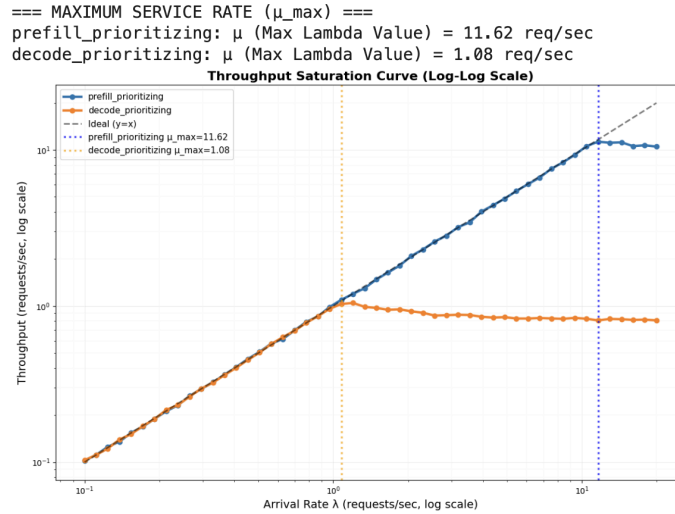
We model an LLM-serving system as a discrete-event simulation with stochastic arrivals, GPU batching, and two execution phases per query: prefill and decode. The simulator captures the interaction between arrival rate, batching, and scheduling policy, and measures their impact on throughput and latency metrics such as TTFT and TBT. The system consists of a single logical GPU worker, a queue of arriving queries, and a scheduler that admits the queries in an active batch. The query model has an arrival time, prompt length L_i , response length B_i , number of prefill tokens, number of decode tokens, time to first token, and completion time. The queries arrive at a Poisson process rate λ .

For building the models, we are using the service model based on the equation below to minimize $E(S(b))$ for all the queries for each of the policies. A request's TTFT is the sum of its queuing time (waiting for a slot) and the total service time taken by all preceding prefill steps (its own and others in the batch). Reducing $S(b)$ directly reduces the prefill duration, improving TTFT. TBT is essentially equal to the $S(b)$ incurred during the decode phase when the batch contains decode requests. If the decode batch is large, $S(b)$ is high, leading to high TBT and a slow streaming experience.

$$S(b) = c + a \cdot \max(0, b - b_0)$$

1. Throughput Saturation and Max Service Rate

The figure below shows the throughput–arrival rate relationship for two scheduling policies on a log-log scale. At low arrival rates ($\lambda < \mu$), both policies closely follow the ideal $y = x$ line, indicating that the system is arrival-limited: every incoming request is processed immediately, and throughput increases linearly with load. As λ increases, the system approaches its maximum service capacity μ , after which throughput saturates and no longer tracks arrivals. For the prefill-prioritizing policy, throughput continues to scale until saturating near $\mu \approx 11.6$ requests/sec, demonstrating efficient batching and high GPU utilization. In contrast, decode-prioritizing saturates much earlier at $\mu \approx 1.08$ requests/sec, reflecting its conservative admission strategy, which preserves decode smoothness but severely limits throughput. To simplify future simulations, we chose a λ of 0.25 as the system will be unsaturated, where the behavior can be interpretable as it is well below the maximum capacity of both models.

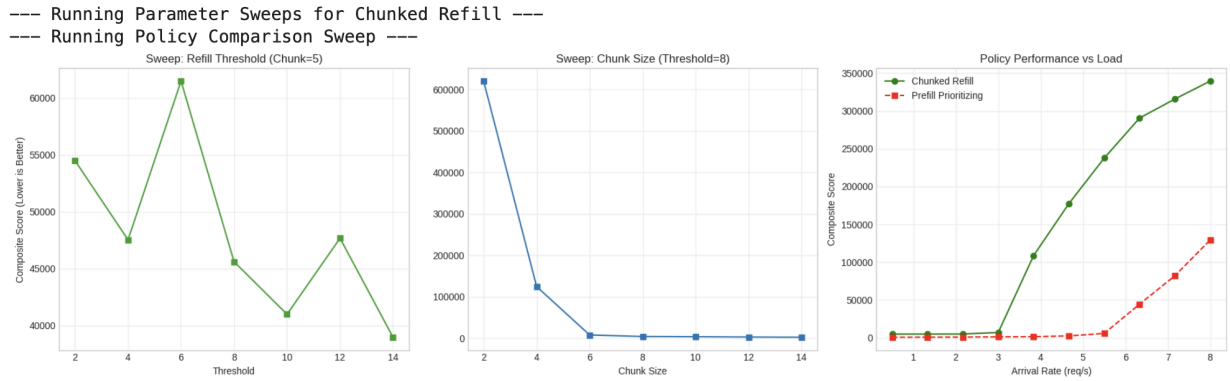


2. Optimization of Chunked Refill Parameters

To evaluate the effectiveness of the chunked refill policy, we conducted a sensitivity analysis over two key parameters: the refill threshold and the prefill chunk size. The first plot shows how the composite latency score varies with the refill threshold (holding chunk size fixed). Performance is poor at very low thresholds, where the GPU frequently waits to accumulate enough work, and also degrades at high thresholds, where refills occur too aggressively, and the policy begins to resemble FCFS prefill prioritization. The lowest composite scores occur around thresholds of 8–10, indicating a balanced tradeoff between utilization and responsiveness.

The second plot examines the impact of chunk size (holding the threshold fixed). Very small chunks incur significant overhead, leading to extremely high latency scores, while performance stabilizes for chunk sizes of 6 tokens and above. Beyond this point, increasing the chunk size yields diminishing returns, suggesting that moderate chunk sizes are sufficient to amortize setup costs without excessively blocking decode iterations.

Finally, the third plot compares the tuned chunked refill policy against the prefill-prioritizing baseline across arrival rates. At low loads ($\lambda \leq 3$), both policies perform similarly, as the system is underutilized and scheduling decisions matter less. As load increases, the chunked refill policy exhibits substantially higher composite latency scores, reflecting its increased overhead and sensitivity to congestion in this configuration. Overall, these results suggest that while chunked refill can be tuned to improve latency balance, its benefits are highly sensitive to parameter choices and workload intensity, and it does not uniformly outperform the simpler prefill-prioritizing baseline across all regimes.



Model Parameters

The parameters of interest that we have built our model with are:

$$\lambda = 0.25$$

$$a = 0.3 \text{ ms/token}$$

$$c = 35 \text{ ms}$$

$$L = 100 \text{ (fixed)}$$

$$b \text{ (fixed)} = 20$$

$$k = 256$$

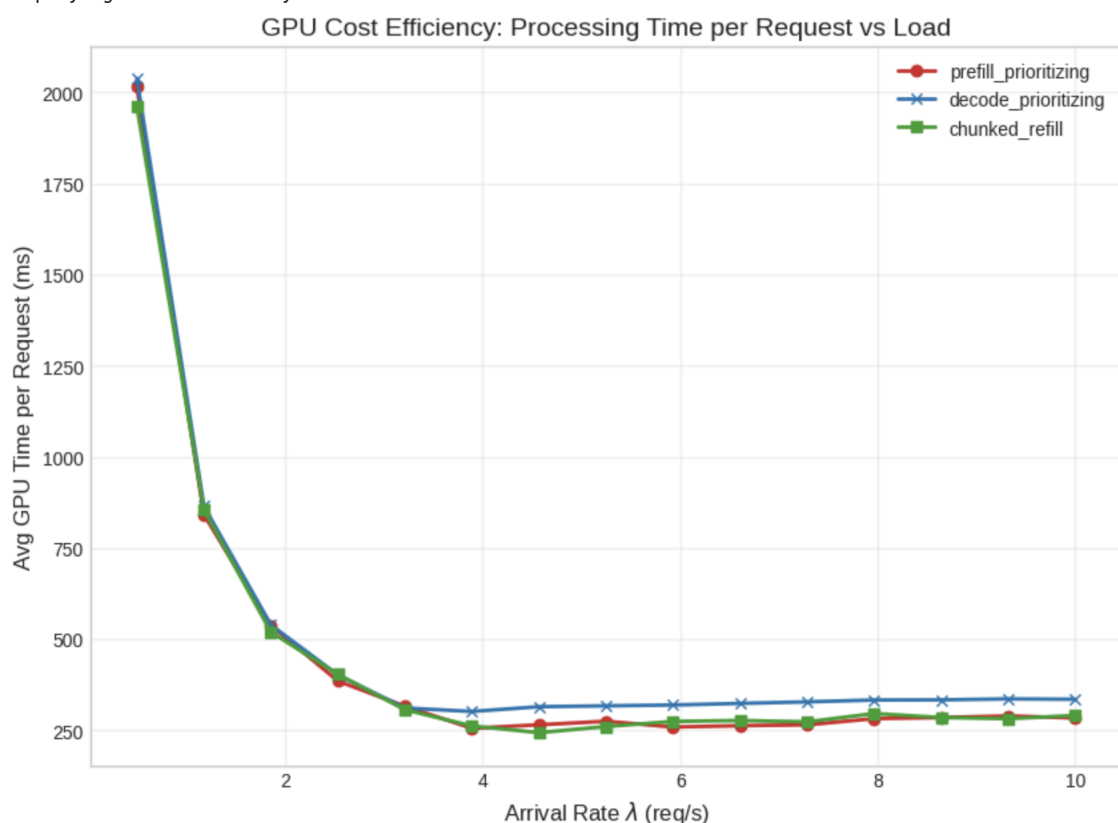
$$b_0 \text{ (given)} = 64$$

$$\text{chunked_refill} = 5$$

Model Analysis

1. GPU Cost Efficiency

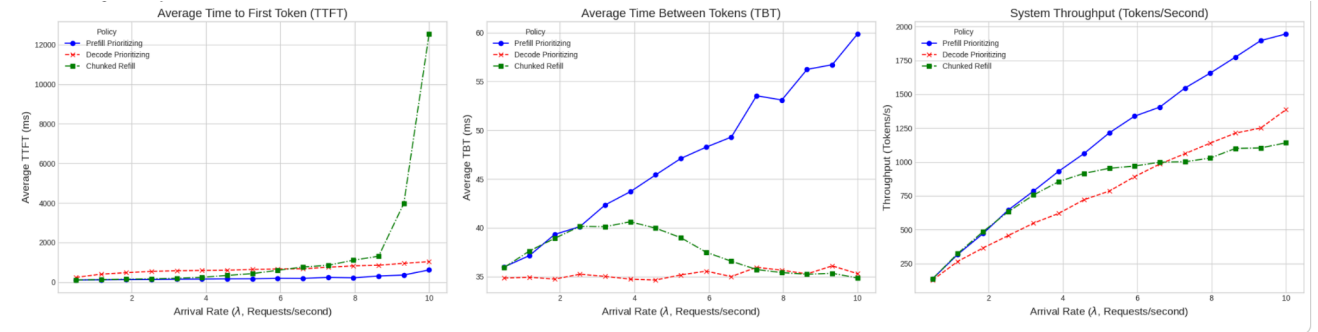
When analyzing the models, we want to look at how much it costs for the computer to process the query. In this case, X is the arrival rate, and Y is the GPU time for each request. In this scenario, we see that prefill prioritizing would be lower, leading to less idle time for the GPU, being the cost-efficient alternative. A decode prioritizing policy would keep answers flowing, which keeps the TBT low, but leads to high cost. Because the batching policy will balance between the two, we can see that the cost will be efficient. We can see from the plot below that the intuition is correct.



2. TTFT & TBT

The figure illustrates how arrival rate λ affects user-facing latency and throughput under prefill-prioritizing, decode-prioritizing, and chunked prefill scheduling policies. At low to

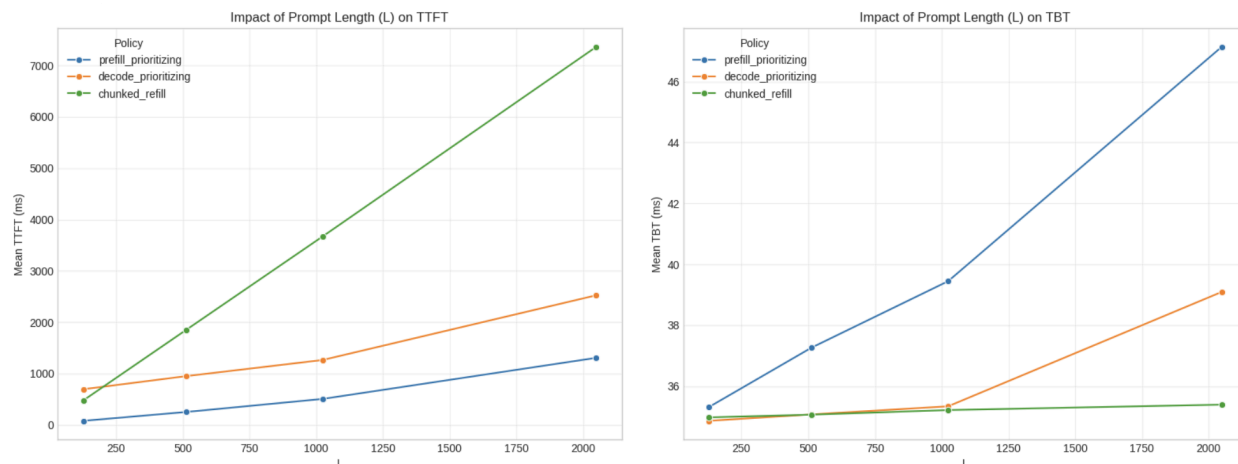
moderate arrival rates, prefill-prioritizing achieves the lowest Time to First Token (TTFT) by aggressively admitting new queries, but this comes at the cost of steadily increasing Time Between Tokens (TBT) as decode iterations are frequently interrupted. Decode-prioritizing shows the opposite behavior: TTFT is consistently higher because new requests must wait for ongoing decodes to finish, but TBT remains low and stable across all arrival rates, resulting in smoother streaming once generation begins. The chunked prefill policy initially strikes a balance between these two extremes, achieving moderate TTFT while significantly reducing TBT relative to prefill-prioritizing by interleaving smaller prefill chunks with decode steps. However, at high arrival rates, chunked prefill exhibits a sharp increase in TTFT, indicating queue buildup as prefill work accumulates faster than it can be serviced. Throughput results further highlight this tradeoff: prefill-prioritizing achieves the highest throughput, decode-prioritizing the lowest, and chunked prefill lies in between, confirming that chunked prefill improves user experience at moderate loads but can suffer stability issues under heavy traffic.



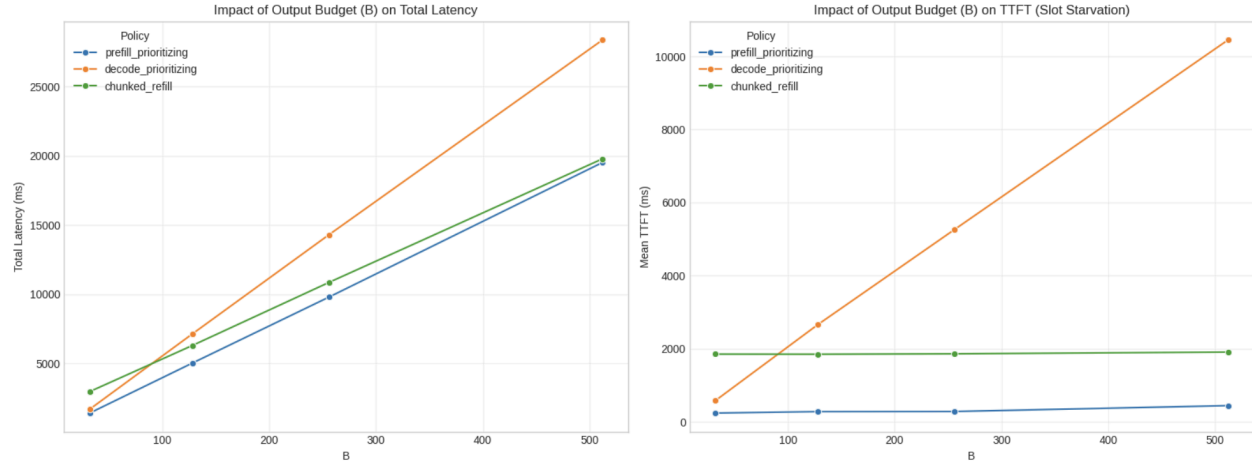
3. Varying Fixed L and B

To evaluate the robustness of the three scheduling policies, we performed a sensitivity analysis by varying the prompt length L . As L increases from 128 to 2048 tokens, clear differences emerge between setup-phase and streaming-phase performance. Under prefill-prioritizing, TTFT grows moderately with L , but TBT increases sharply, indicating that long prefills monopolize GPU resources and cause decode stalls for active users. Decode-prioritizing shows the opposite behavior: TBT remains low and nearly flat across all prompt lengths, since decode work is never interrupted, but TTFT increases substantially because new requests must wait for ongoing decodes to complete. The chunked-refill policy mitigates both effects by splitting large prefills into smaller chunks. As a result, it significantly reduces the growth of TBT compared to prefill-prioritizing while keeping TTFT lower than decode-prioritizing for moderate prompt

lengths. However, TTFT still grows linearly with L , reflecting the unavoidable cost of processing large prompts. Overall, these results confirm that chunked prefill improves fairness and streaming smoothness for long prompts, but does not fully eliminate the latency impact of very large setup workloads.



The figures illustrate an important stability tradeoff as the output budget B increases. As B grows, each query occupies a decode slot for a longer period, increasing contention in the system. Under prefill-prioritizing scheduling, both total latency and TTFT increase roughly linearly with B , indicating stable behavior: although longer responses take more time, new requests are still admitted regularly and do not experience severe slot starvation. In contrast, decode-prioritizing exhibits rapidly increasing TTFT as B grows, since long-running decodes block admission of new requests, causing severe queuing delays. The chunked refill policy initially moderates this effect by interleaving decode with partial prefill work; however, once B exceeds approximately 250 tokens, TTFT for chunked refill plateaus at a high level, reflecting accumulated prefill backlog and scheduling overhead from managing many small chunks. This suggests that while chunked refill improves fairness and smoothness at moderate response lengths, its benefits diminish for long conversations, where simpler FCFS batching handles extended decode workloads more robustly.



4. Throughput, TTFT, and TBT Time Series Analysis

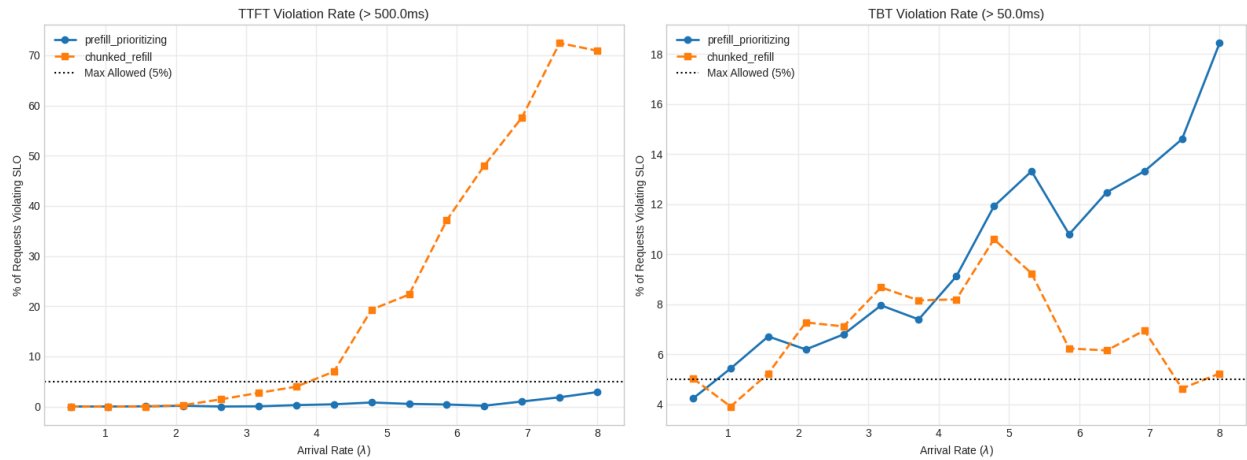
We examine how system performances evolve over time under each scheduling policy. To observe this, we plot arrival-normalized throughput, mean TTFT, and mean TBT as functions of time since steady state.



Across all three policies, throughput remains relatively stable over time, fluctuating over a similar mean value. This aligns with expected behavior, indicating that throughput is primarily constrained by GPU capacity and batching efficiency as opposed to scheduling order. The drop in throughput on the right-hand side of the plot indicates the end of the simulation rather than a policy failure. Overall, we notice that scheduling decisions have little impact on throughput in the long run.

The TTFT time series shows that under decode-prioritizing, the mean TTFT increases over time. Because decode tasks are always completed before admitting new work, newly arriving queries accumulate in the queue. This leads to unbounded delays before the first token is produced. Prefill prioritization achieves a much lower TTFT initially, but displays gradual growth over time as the system becomes congested. Chunked prefill seems to exhibit similar behavior to prefill prioritizing, even displaying less growth than prefill prioritization later on in the time series. The TBT time series reveals the tradeoff between different policies. Decode prioritization maintains consistently low and stable TBT, while prefill prioritization shows the highest TBT with the largest variability. Chunked prefill lies between these extremes, with a lower TBT relative to prefill prioritization, while still accepting higher TBT than decode prioritization. Decode prioritizing provides smooth streaming, but leads to unstable TTFT, while prefill prioritizing minimizes initial response time at the cost of decode interruptions. Chunked prefill offers a balance between the two and displays stable behavior over time. These plots confirm that chunked prefill is the best-performing policy under sustained steady-state operation.

5. SLO Violation Rates



Our simulation reveals a trade-off between minimizing initial wait times and ensuring a smooth streaming experience, governed by the choice of scheduling policy. The prefill prioritizing (baseline) policy excels at meeting the TTFT objective, maintaining a near-zero violation rate even as arrival rates approach saturation (λ approx. 8.0 req/s). By treating prefill requests as high-priority tasks, it ensures new users are admitted immediately. However, this comes at a severe cost to the TBT SLO. As load increases, the large prefill computations act as "blocking" events for the GPU, causing the TBT violation rate to climb linearly, reaching nearly 18% at high load. This confirms the [Head-of-Line Blocking](#) phenomenon: every new arrival forces all active users to suffer a significant pause in their output generation. In contrast, chunked-refill effectively inverts this dynamic. By breaking large prefill jobs into smaller, non-blocking chunks, it prioritizes the continuity of existing streams. Consequently, chunked-refill maintains a significantly lower TBT violation rate, staying close to the 5% tolerance threshold far longer than the baseline. The trade-off is a dramatic degradation in TTFT compliance. As new requests must wait for available "slots" to process their prefill chunks over multiple iterations, the TTFT violation rate skyrockets to over 70% at high loads. This demonstrates that while chunked-refill successfully eliminates generation stalls, it does so by imposing a "latency tax" on new arrivals, making it an ideal choice for applications where stream smoothness is paramount over instantaneous start times.

Conclusion

We explored three different scheduling policies, prefill-prioritizing, decode-prioritizing, and chunked-refill, to understand how different scheduling decisions affect the efficiency of LLM query serving. Our results confirm three core claims introduced at the beginning of this paper. Prefill-prioritizing achieves a lower average TTFT by aggressively admitting new queries, but does so at the expense of a higher average TBT due to frequent interruptions of decoding. Decode-prioritizing exhibits the opposite tradeoff. By completing ongoing decode tasks before admitting new work, it maintains smoother token streaming and lower TBT, but causes TTFT to grow as newly arriving queries accumulate in the queue. Finally, chunked-refill breaks large prefill workloads into smaller units that can be interleaved with decode steps. As a result, it narrows the gap between TTFT and TBT, consequently providing a more balanced allocation of resources between prompt processing and response generation.

However, response latency alone does not fully characterize system performance. Our experiments show that each scheduling strategy also induces distinct hardware utilization and cost profiles that vary with queue depth and batching behavior. Prefill-prioritizing maximizes throughput and minimizes total completion time under heavy load, but leads to inefficient decode scheduling and degraded streaming quality. Decode-prioritizing ensures predictable, low-variance decoding at the cost of underutilized prefill capacity and growing admission delays. Chunked-refill offers a practical compromise, reducing decode stalls while avoiding the unbounded TTFT growth of decode-prioritizing policies, though it introduces additional overhead from repeated prefill setup. Taken together, these findings highlight that no single scheduling policy dominates across all workloads. Instead, the optimal choice depends on user expectations and service-level objectives. Interactive applications benefit from smoother streaming and should favor chunked-refill or decode-prioritizing strategies, while throughput-oriented or offline workloads are better served by prefill-prioritizing scheduling.

Technical Appendix

1. Building the Model

To build the model, we built a query class with the following parameters:

```
class Query:
    def __init__(self, q_id, arrival_time, prompt_len, output_len):
        self.q_id = q_id
        self.arrival_time = arrival_time
        self.L = prompt_len
        self.B = output_len
        self.prefill_tokens_processed = 0
        self.decode_tokens_generated = 0
        self.prefill_completed = False
        self.first_token_time = None
        self.completion_time = None
        self.decode_token_times = []
        self.admission_time = None
```

We have the following helper functions to build out the model:

calculate_service_time(batch_token_load, params):

Computes how long one GPU “iteration” takes for the current batch using the service-time model equation, where c , a , and b_0 come from `params`, and then adds randomness by sampling an exponential time around that mean.

def get_new_admissions(queue, active_batch, current_time, params):

Moves newly-arrived queries from the waiting queue into `active_batch` based on the scheduling policy in `params['policy']` and the concurrency cap `params['max_requests']` (with `decode_prioritizing` blocking admissions whenever any active request is in decode).

def get_metrics_df(requests):

Turns completed Query objects into a pandas DataFrame of user metrics—TTFT, TBT, and Total_Latency—computed from each request’s recorded timestamps (`arrival_time`, `first_token_time`, `decode_token_times`, `completion_time`).

def run_simulation(num_requests, params, warmup):

Runs the full discrete-event simulation: generates arrivals using `params['lambda']`, enforces GPU capacity using `params['K']`, applies chunking using `params['policy'] + params['chunk_size']`, advances time using `calculate_service_time`, updates per-query state, and returns completed requests after discarding warmup.

Below is the code to look at the simulation running in depth:

```
# --- Main Simulation Loop (With Token-Level Chunking) ---
def run_simulation(num_requests, params, warmup=100):
    # 1. Generate Arrivals
    inter_arrivals = np.random.exponential(1.0/params['lambda'],
num_requests)
    arrival_times = np.cumsum(inter_arrivals)

    queue = collections.deque([
        Query(i, arrival_times[i], params.get('fixed_L', 100),
params.get('fixed_B', 20))
        for i in range(num_requests)
    ])

    active_batch = []
    completed_requests = []
    current_time = 0.0

    while len(completed_requests) < num_requests:
        # A. Admission Phase
        new_reqs = get_new_admissions(queue, active_batch, current_time,
params)
        for req in new_reqs: req.admission_time = current_time
        active_batch.extend(new_reqs)

        if not active_batch:
            if queue:
                current_time = queue[0].arrival_time
                continue
            else:
                break

        # B. Scheduling Phase (Token Calculation)
```

```

K = params['K']

# Apply Chunk Limit ONLY if policy is chunked_refill
if params.get('policy') == 'chunked_prefill':
    max_token_chunk = params.get('chunk_size', 64)
else:
    max_token_chunk = 2048 # Effectively infinite

prefill_reqs = [r for r in active_batch if not r.prefill_completed]
decode_reqs = [r for r in active_batch if r.prefill_completed]

batch_load = len(decode_reqs) # Decodes take 1 slot each
remaining_capacity = K - batch_load

# Plan work for this step
work_plan = {}

if prefill_reqs and remaining_capacity > 0:
    # Fair share of the GPU capacity
    fair_share = max(1, remaining_capacity // len(prefill_reqs))

    # Determine limit: Min of (Fair Share, Hard Chunk Limit)
    tokens_per_request_limit = min(fair_share, max_token_chunk)

    for req in prefill_reqs:
        remaining_work = req.L - req.prefill_tokens_processed
        actual_work = min(remaining_work, tokens_per_request_limit)
        work_plan[req.q_id] = actual_work
        batch_load += actual_work

# C. Execution Phase
step_duration = calculate_service_time(batch_load, params)
current_time += step_duration

# D. Update Phase
still_active = []

# Update Prefills
for req in prefill_reqs:
    if req.q_id in work_plan:

```

```

        req.prefill_tokens_processed += work_plan[req.q_id]
        if req.prefill_tokens_processed >= req.L:
            req.prefill_completed = True
            req.first_token_time = current_time
        still_active.append(req)

    # Update Decodes
    for req in decode_reqs:
        req.decode_tokens_generated += 1
        req.decode_token_times.append(current_time)
        if req.decode_tokens_generated >= req.B:
            req.completion_time = current_time
            completed_requests.append(req)
        else:
            still_active.append(req)

    active_batch = still_active

    return completed_requests[warmup:]

```

2. Steady State Considerations

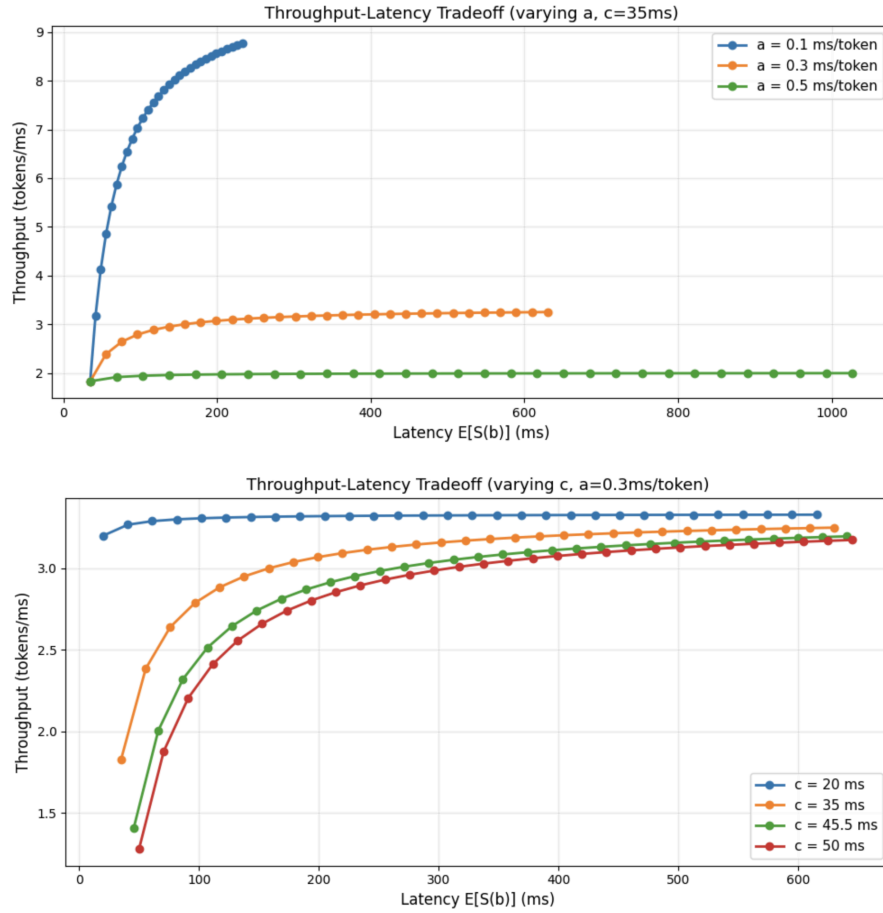
We are generating the queries with a steady state approach by running the simulation 10,000 times, then discarding the first 2,000 as a warmup.

```
reqs = run_simulation(num_reqs, p, warmup=2000)
```

3. Hardware Characteristics

Before generating each of the policies, we want to find an optimal a and c . Currently, the Llama-3 model uses a $c = 20\text{-}50$ ms, and $a = 0.1\text{-}0.5$ ms/token. C is the fixed setup cost per batch. This cost is paid every time the GPU runs a batch. A is the marginal cost per token and calculates how efficiently the GPU processes additional tokens. To find optimal a and c values, we used the Llama-3 range, and plotted various a and c values to analyze its throughput-latency tradeoff. For a , we chose 0.3 as it presents clear throughput gains from batching, while diminishing returns at high latencies. For c , we chose 35 ms, as we are also able to balance the throughput-latency tradeoff, and make the scheduler decision matter. The a and c parameters

dictate the computer system itself, so it does not necessarily matter in the long run, where the scheduling and batching are more important. The shapes of the curves do not change when the values change, as it is driven by queueing, and not the GPU. Furthermore, policy ranking does not change across all experiments as prefill-prioritizing leads to higher throughput, worse TTFT, and decode prioritizing leads to better TTFT, lower throughput, and the order stays the same even through differing a and c .



4. Validation

```
... --- M/M/1 Validation: Analytical Results ---
Arrival Rate (lambda): 0.5
Service Rate (mu): 1.0
Utilization (rho): 0.5000
Theoretical Avg Waiting Time (Wq): 1.0000 (Time units)
Theoretical Avg System Time (W): 2.0000 (Time units)
Theoretical Avg Jobs in Queue (Lq): 0.5000 (Jobs)
Theoretical Avg Jobs in System (L): 1.0000 (Jobs)

--- M/M/1 Validation: Simulation Run ---

Simulated Avg Waiting Time (Wq_sim): 1.0082
Simulated Avg System Time (W_sim): 2.0128
Simulated Avg Jobs in Queue (Lq_sim): 0.5041
Simulated Avg Jobs in System (L_sim): 1.0064

--- M/M/1 Validation: Error Check (Should be < 5%) ---
Wq Error: 0.82%
W Error: 0.64%
Lq Error: 0.82%
L Error: 0.64%
```

The simulation model was validated against the well-established M/M/1 queueing theory to ensure the accuracy of its fundamental event-handling logic. This validation case utilized parameters corresponding to the M/M/1 system: no batching ($B=1$), no setup time ($c=0$), and strictly exponential service and inter-arrival times. The results show excellent agreement with the analytical formulas for all key performance indicators. Specifically, the simulated average waiting time (Wq) and average system time (W) deviated from their theoretical values by only 0.82% and 0.64%, respectively. The derived queue lengths (Lq_{sim} and L_{sim}) exhibited similarly minimal errors, confirming that the simulation accurately models the behavior of a single-server queue operating under Poisson arrivals and exponential service times. With an error margin consistently below 1.0%, the model is validated and deemed statistically reliable for use in analyzing the complex effects of batching and setup times as intended by the project scope.